The default `application.rb` file has instructions on how to add locales from another directory and how to set a different default locale. Just uncomment and edit the specific lines.

```
# The default locale is :en and all translations from
config/locales/*.rb,yml are auto loaded.
# config.i18n.load_path += Dir[Rails.root.join('my', 'locales',
'*.{rb,yml}').to_s]
# config.i18n.default_locale = :de
```

## 2.2 Optional: Custom I18n Configuration Setup

For the sake of completeness, let's mention that if you do not want to use the `application.rb` file for some reason, you can always wire up things manually, too.
To tell the I18n library where it can find your custom translation files you can specify the load path anywhere in your application - just make sure it gets run before any translations are actually looked up.
You might also want to change the default locale. The simplest thing possible is to put the following into an initializer:

```
# in config/initializers/locale.rb

# tell the I18n library where to find your translations
I18n.load_path += Dir[Rails.root.join('lib', 'locale', '*.{rb,yml}')]

# set default locale to something other than :en
I18n.default_locale = :pt
```

## 2.3 Setting and Passing the Locale

If you want to translate your Rails application to a **single language other than English** (the default locale), you can set I18n.default_locale to your locale in `application.rb` or an initializer as shown above, and it will persist through the requests.
However, you would probably like to **provide support for more locales** in your application. In such case, you need to set and pass the locale between requests.
You may be tempted to store the chosen locale in a *session* or a *cookie*. However, **do not do this**. The locale should be transparent and a part of the URL. This way you won't break people's basic assumptions about the web itself: if you send a URL to a friend, they should see the same page and content as you. A fancy word for this would be that you're being*RESTful*. Read more about the RESTful approach in Stefan Tilkov's articles. Sometimes there are exceptions to this rule and those are discussed below.

The *setting part* is easy. You can set the locale in a `before_action` in the `ApplicationController`like this:

```
before_action :set_locale

def set_locale
  I18n.locale = params[:locale] || I18n.default_locale
end
```

This requires you to pass the locale as a URL query parameter as
in `http://example.com/books?locale=pt`. (This is, for example, Google's approach.)
So `http://localhost:3000?locale=pt` will load the Portuguese localization,
whereas `http://localhost:3000?locale=de` would load the German localization, and so on. You may skip the next section and head over to the **Internationalize your application** section, if you want to try things out by manually placing the locale in the URL and reloading the page.
Of course, you probably don't want to manually include the locale in every URL all over your application, or want the URLs look differently, e.g. the